

FS Challenge

Full Stack Engineering Homework Assignment

Estimated Time

2-3 hours

1. Objective

Build a small application that allows users to search and browse information from a public API.

The goal of this assignment is not to create a feature-rich product, but to demonstrate:

- Clean architecture
- Separation of concerns
- SOLID principles
- API design
- Code maintainability
- Scalability and extensibility
- Frontend and backend best practices

2. Functional Requirements

2.1. Backend (NestJS + GraphQL)

Build a GraphQL API that acts as an abstraction layer over a public API of your choice.

Examples:

- Star Wars API (SWAPI)

- PokeAPI
- Open Library API
- REST Countries API
- SpaceX API

No API keys should be required.

Your GraphQL API must:

Queries

Provide at least:

- List entities
- Fetch a single entity
- Filter/search entities

Example:

```
query {  
  getCountries(search: "uk")  
}
```

2.2. Responsibilities

The backend should:

- Fetch data from the external API
- Map external models into internal domain models
- Hide external API implementation details
- Expose a clean GraphQL contract

2.3. Frontend (React/Next.js or Flutter)

Build a simple client application consuming your GraphQL API.

The application should allow users to:

- View a list of entities

- Search/filter entities
- View entity details

Example:

Country Explorer

- Countries list
- Search by name
- Country details page

3. Technical Requirements

3.1. Architecture

Both frontend and backend must demonstrate:

- Clear separation of concerns
- Layered architecture
- Dependency inversion where appropriate
- Easy extensibility

3.2. SOLID Principles

The codebase should clearly demonstrate understanding of:

- Single Responsibility Principle
- Open/Closed Principle
- Dependency Inversion Principle

Perfect implementation is not expected, but architectural decisions should support future growth.

4. Deliverables

Post your project on your github account and share the repo URL to alexandru.rusu@pynea.com & juned.khan@pynea.com

4.1. Source Code

Git repository containing:

- Frontend
- Backend

4.2. README

Readme should include:

Architecture Overview

Explain:

- Project structure
- Architectural decisions
- Design patterns used

Trade-offs

Describe:

- What was intentionally simplified
- What would be improved with more time

Running the Project

Instructions to:

- Install dependencies
- Start backend
- Start frontend

5. Bonus Enhancements (Optional)

Feel free to go beyond the baseline and include anything you consider necessary for a modern, production-ready application.